

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Towards Dev · [Follow publication](#)

Eliminating Page-Walk Stalls Using Huge Pages: Low Latency C++

Stop wasting CPU cycles on page table walks — a guide to implementing huge pages in C++.

5 min read · May 22, 2026



Sagar

Following ▾

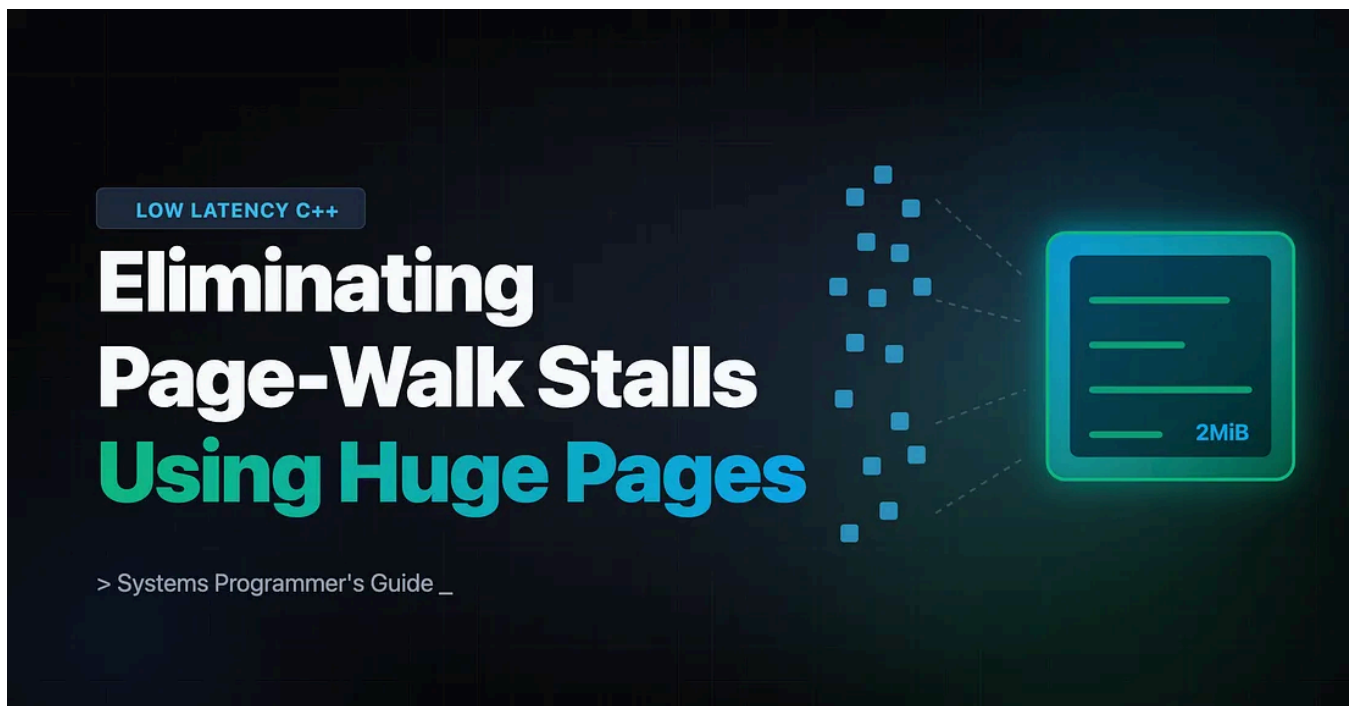


Listen



Share

⋮ More



In my previous post, *[Systems Programmer's Guide to the TLB](#)*, we dug into how the Translation Lookaside Buffer works and why TLB misses are so devastating to low-

latency applications. At the end of that article, I promised we would cover Huge Pages in a separate post — so it is time to make good on that promise.

When you are trying to squeeze every last nanosecond of performance out of modern x86-64 hardware, standard memory management often becomes a silent bottleneck. It doesn't matter how optimized your data structures are if your CPU is wasting precious cycles just trying to translate virtual addresses before it can even touch your data.

In this Post, we are going to explore **Huge Pages**.

Huge Pages and The Math Behind this

By default, the 64-bit x86 architecture and Linux handle memory in **4KiB** chunks called **pages**. Every memory access requires translating a virtual address to a physical address via the page table, which may trigger a multi-level page walk involving several dependent memory accesses.

While the **TLB** caches these translations, its size is strictly limited. Across its multi-level TLB hierarchy, a modern server CPU may cache only a few thousand translations per core. For a rough calculation, let's assume the CPU can cache around 2000 TLB translations per core.

Let's look at the math for how much memory (the “working set”) the TLB can cover before you suffer a painful TLB miss.

Using standard 4KiB pages:

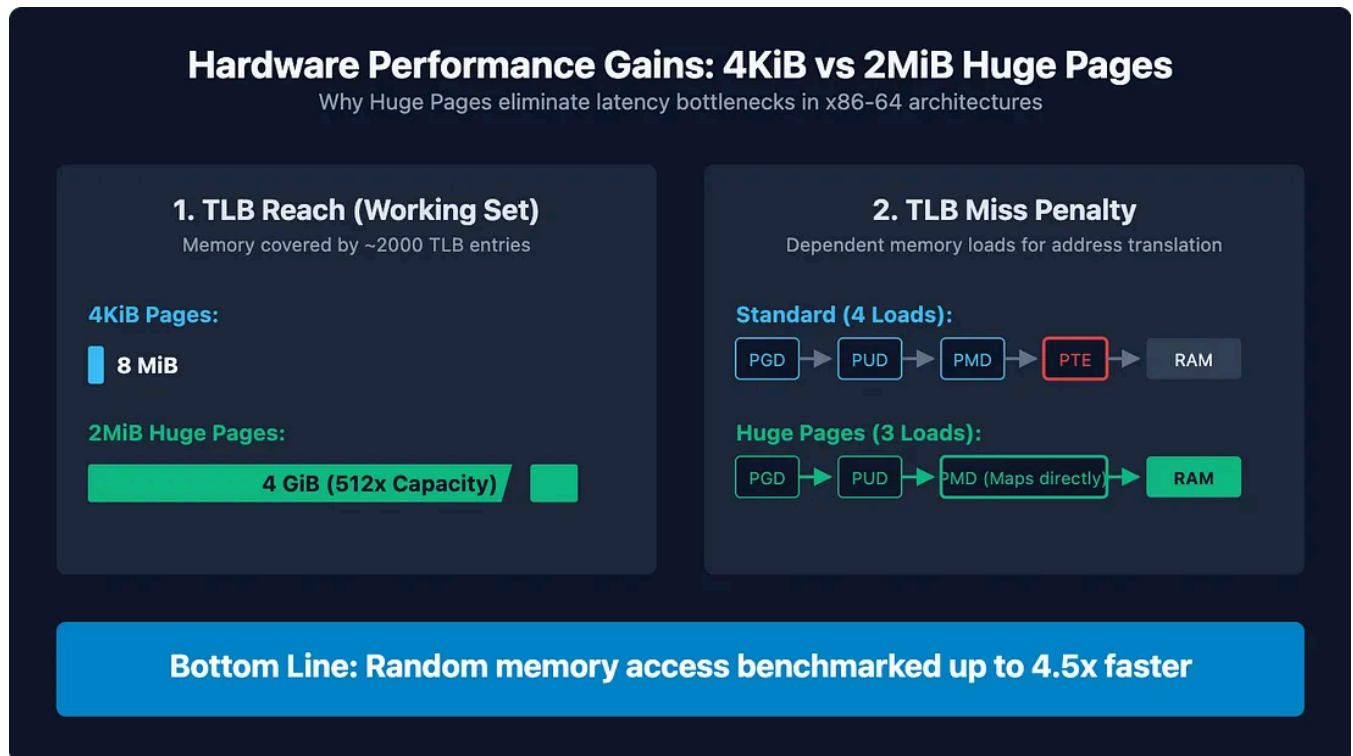
$$\text{Coverage}(4\text{KiB}) = 2000 \text{ entries} \times 4\text{KiB} \approx 8\text{MiB}$$

Using 2MiB Huge Pages:

$$\text{Coverage}(2\text{MiB}) = 2000 \text{ entries} \times 2\text{MiB} \approx 4\text{GiB}$$

By switching to 2MiB pages (or even 1GiB gigantic pages), one page covers 512 times more memory. This reduces page-table depth, shrinks page-table overhead, and allows the TLB to map gigabytes of memory instead of just a few megabytes.

Huge pages reduce page-walk depth, lowering translation overhead on TLB misses.



Two Paths to Huge Pages: THP vs. Hugetlbfs

Linux provides two primary mechanisms to allocate huge pages, each with unique trade-offs. Remember, the OS must find 512 *contiguous* physical 4KiB pages to form a **single 2MiB huge page**, which can be difficult if system memory is highly fragmented.

1. Transparent Huge Pages (THP)

Introduced over 10 years ago, THP allows the kernel (via the **khugepaged** daemon) to swap 4KiB pages for huge pages automatically in the background.

- **always mode:** The OS aggressively attempts to use huge pages for most allocations. It's easy, but you lose control over what gets promoted and when.
- **madvise mode:** You explicitly tell the kernel which memory regions should be backed by huge pages using the **madvise()** system call.

Caveat: When memory is fragmented, the OS will defragment RAM by moving running programs around. This causes latency spikes. If your application is extremely latency-sensitive, THP might introduce unwanted jitter.



Pro-tip for low-latency systems: Use `madvise` mode, and configure the OS defrag behavior via `/sys/kernel/mm/transparent_hugepage/defrag`. Set it to `defer` or disable it entirely to prevent unpredictable jitter.

2. Hugetlbfs (Explicit Huge Pages)

This is a **pseudo-filesystem** that manages a dedicated pool of huge pages.

- Pages here are reserved exclusively for **hugetlbfs** and cannot be swapped to disk.
- Allocations from a pre-allocated pool are lightning-fast.

Caveat: If your program requests a huge page and the pool is maxed out, the OS sends a **SIGBUS** signal, instantly killing the process. This frequently catches developers off guard during copy-on-write operations (like after calling `fork()`).

Rule of thumb: Avoid **hugetlbfs** if your process spawns child processes.

Here is how you can explicitly request huge pages using both methods in a standard C++ application.

```
#include <iostream>
#include <sys/mman.h>
#include <unistd.h>
#include <cstring>

// Size must be a multiple of 2MiB (2 * 1024 * 1024 bytes)
constexpr size_t ALLOC_SIZE = 2 * 1024 * 1024;

void allocate_with_hugetlbfs() {
    // MAP_HUGETLB explicitly requests pages from the hugetlbfs pool
```

```

void* ptr = mmap(nullptr, ALLOC_SIZE, PROT_READ | PROT_WRITE,
                 MAP_PRIVATE | MAP_ANONYMOUS | MAP_HUGETLB, -1, 0);

if (ptr == MAP_FAILED) {
    std::cerr << "Hugetlbfs allocation failed. Is the pool configured?\n";
    return;
}
std::cout << "Successfully allocated 2MiB via hugetlbfs.\n";
munmap(ptr, ALLOC_SIZE);
}

void allocate_with_thp_madvise() {
    // Standard anonymous mapping
    void* ptr = mmap(nullptr, ALLOC_SIZE, PROT_READ | PROT_WRITE,
                     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);

    if (ptr != MAP_FAILED) {
        // Advise the kernel to use Transparent Huge Pages for this specific region
        if (madvise(ptr, ALLOC_SIZE, MADV_HUGEPAGE) == 0) {
            std::cout << "Successfully advised kernel to use THP.\n";
        } else {
            std::cerr << "madvise failed to set MADV_HUGEPAGE.\n";
        }
        munmap(ptr, ALLOC_SIZE);
    }
}

int main() {
    allocate_with_hugetlbfs();
    allocate_with_thp_madvise();
    return 0;
}

```

(Note: If you use glibc 2.35 or newer, you don't even need custom `mmap` calls for standard allocations; you can just set the `glibc.malloc.hugetlb` tunable!)

Measuring the Impact

You shouldn't guess when it comes to performance. Before and after applying huge pages, you need to measure your cache hits and misses directly from the hardware.

You can use the Linux `perf` tool to count exact data TLB (dTLB) loads and misses for your application:

```
# Measure data TLB hardware events
sudo perf stat -e dTLB-loads,dTLB-load-misses ./your_program

# Alternatively, check general page-fault behavior
sudo perf stat -e page-faults ./your_program
```

To verify if your system is actually allocating and utilizing Huge Pages under the hood, you can peek at the kernel's memory info:

```
# Check how many huge pages are allocated, free, and reserved
grep Huge /proc/meminfo
```

Don't Forget the Hardware Nuances

Before you rewrite your allocator, remember that CPU caches are split. The TLB has multiple levels (L1 and L2) separated into instruction (iTLB) and data (dTLB) queues.

On an Intel Cascade Lake CPU, the L1 data TLB might only have 32 entries specifically reserved for 2MiB pages, alongside 64 entries for 4KiB pages. Because of these hardware-level segregations, converting *every single page* to a huge page might actually evict hot entries from your L1 TLB. Sometimes, keeping your absolute hottest memory paths on 4KiB pages yields the lowest possible latency.

Another important consideration is **NUMA locality**. Huge pages do not solve poor NUMA placement. In low-latency systems, NUMA locality and CPU affinity are often just as important as TLB optimization.

Closing Thoughts

Huge pages aren't a silver bullet, but for memory-intensive, low-latency applications, they are one of the most powerful OS-level optimizations at your disposal.

By flattening the page walk and massively expanding your TLB reach, you reclaim CPU cycles that were otherwise lost to the void of address translation.

Just remember to always profile your workloads with **perf**, choose your allocation strategy carefully, and never guess when you can measure. Happy optimizing!

Programming

Software Development

Low Latency

C Plus Plus Language

Systems Programming



Follow

Published in Towards Dev

10.1K followers · Last published 22 hours ago

A publication for sharing projects, ideas, codes, and new theories.



Following ▾

Written by Sagar

169 followers · 6 following

System Programmer | C++ | Linux System Programming | Rust | Python | Cloud Computing | DevOps | Low Latency Backends